

1. Logika cyfrowa

| Układ                       | Formuła / sygnatura                                                 | Opis                                                                                                                              |
|-----------------------------|---------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Sumatory                    |                                                                     |                                                                                                                                   |
| Półsumator                  | $s = x \oplus y$<br>$c = x \& y$                                    | Dodaje dwa bity. <b>s</b> to suma, <b>c</b> to bit przeniesienia (carry).                                                         |
| Pełny sumator (FA)          | $s = x \oplus y \oplus ci$<br>$co = x \& y \mid ci \& (x \oplus y)$ | Jak półsumator, ale przyjmuje też przeniesienie wejściowe <b>ci</b> .                                                             |
| Sumator n-bitowy            | $n \times FA$                                                       | Kaskadowo wyjście $C_i$ trafia na wejście kolejnego FA. Ostatnie <b>co</b> to Carry Out całości.                                  |
| Układy kombinacyjne         |                                                                     |                                                                                                                                   |
| Dekoder                     | $n \rightarrow 2^n$                                                 | Wejście koduje liczbę $k$ . Na wyjściu zapalony jest bit $k$ .                                                                    |
| Multiplekser                | $2^n + n \rightarrow 1$                                             | $2^n$ bitów danych + $n$ bitów sterujących $S$ . Na wyjściu zapalony $S$ -ty bit danych.                                          |
| ALU                         | $A, B, f \rightarrow C$                                             | Dekoder na bitach $f$ wybiera operację np. $A + B$ , multipleksowanie wyników bramkami AND/OR.                                    |
| Układy sekwencyjne (pamięć) |                                                                     |                                                                                                                                   |
| Przerzutnik S-R             | <b>S</b> - set, <b>R</b> - reset                                    | Przechowuje 1 bit ( $Q$ ). <b>S=1</b> ustawia $Q = 1$ , <b>R=1</b> zeruje, <b>S=R=0</b> trzyma stan. Dwa sprzężone <b>NOR</b> -y. |
| Sterowany poziomem          | aktywny gdy <b>CLK</b> = 1                                          | Wejścia <b>S</b> / <b>R</b> zAND-owane z zegarem.                                                                                 |
| Sterowany zboczem           | zbocze 1 $\rightarrow$ 0                                            | Dwa przerzutniki poziomowe w kaskadzie. Stan przepisuje się w momencie zmiany zegara.                                             |

2. Typy danych

| Typ    | char | short | unsigned | int | long | float | double | void* |
|--------|------|-------|----------|-----|------|-------|--------|-------|
| x86-64 | 1    | 2     | 4        | 4   | 8    | 4     | 8      | 8     |

3. Rozszerzanie, obcinanie i przesunięcia

|                    |                                               |
|--------------------|-----------------------------------------------|
| $x \ll k$          | $x \cdot 2^k$ (sgn./uns.)                     |
| $u \gg k$          | $\lfloor u/2^k \rfloor$ - logiczne (zera)     |
| $x \gg k$          | arytmetyczne (kopia MSB), zaokr. do $-\infty$ |
| $x/2^k$ do 0       | $(x + (1 \ll (k-1))) \gg k$                   |
| Strength reduction | $x * 24 \rightarrow (x \ll 5) - (x \ll 3)$    |

4. Operacje i endianness

|                                                                                                                                                                                                                                  |                                             |                       |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------|-----------------------|
| Negacja U2                                                                                                                                                                                                                       | <code>-x = ~x+1 ; -TMin=TMin , -0=0</code>  |                       |
| Wykr. nadmiaru ( <code>s=x+y</code> )                                                                                                                                                                                            | <code>((s^x)&amp;(s^y))&gt;&gt;(w-1)</code> |                       |
| Maska / abs                                                                                                                                                                                                                      | <code>m=x&gt;&gt;31; abs=(x^m)-m</code>     |                       |
| <code>c ? a : b</code>                                                                                                                                                                                                           | <code>b ^ (-c &amp; (a ^ b))</code>         |                       |
| Promocja w C: <code>unsigned</code> i <code>int</code> w jednym wyrażeniu/porównaniu<br><code>&lt; &gt; ==</code> ) → <code>int</code> promowany do <code>unsigned</code> (bity bez zmian, <code>-1</code> → <code>Max</code> ). |                                             |                       |
| Schemat                                                                                                                                                                                                                          | Najniższy adres                             | <code>0x0123</code>   |
| Big Endian                                                                                                                                                                                                                       | MSB                                         | <code>[01][23]</code> |
| Little Endian                                                                                                                                                                                                                    | LSB                                         | <code>[23][01]</code> |

5. Zmiennoprzecinkowe (IEEE 754)

$V = (-1)^s \times M \times 2^E$       Bias =  $2^{k-1} - 1$

| Format | bity | s | exp ( $k$ ) | frac ( $n$ ) |
|--------|------|---|-------------|--------------|
| float  | 32   | 1 | 8           | 23           |
| double | 64   | 1 | 11          | 52           |

Kategorie wartości

| Kategoria      | exp                              | Własności                                                                                                                                                    |
|----------------|----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Znormalizowane | $!= 0 \dots 0$<br>$!= 1 \dots 1$ | $E = \text{exp} - \text{Bias}$ . $M = 1.\text{frac}$ .<br>Najgęściej ułożone wokół 0.                                                                        |
| Zdenormaliz.   | $== 0 \dots 0$                   | $E = 1 - \text{Bias}$ . $M = 0.\text{frac}$ .<br>Reprezentują $\pm 0.0$ (frac = 0) i liczby bardzo bliskie zera.                                             |
| Specjalne      | $== 1 \dots 1$                   | $\text{frac} == 0 \rightarrow \pm \infty$ (nadmiar, dzielenie przez 0).<br>$\text{frac} != 0 \rightarrow \text{NaN}$ (np. $\sqrt{-1}$ , $\infty - \infty$ ). |

Zaokrąglenie GRS i Round-to-Even

Domyślny tryb to **Round-to-Even** (zapobiega błędowi statycznemu przy sumowaniu).

|                                                                                                                                                                                |         |                                                                   |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|-------------------------------------------------------------------|
| <div>... x x <b>G</b>    <b>!</b>    <b>R</b> <b>S</b> <b>S</b> <b>S</b> ...</div> <div><b>G</b> - ostatni zachowany, <b>R</b> - pierwszy usunięty, <b>S</b> - 0R reszty</div> |         |                                                                   |
| Warunek                                                                                                                                                                        | Ułamek  | Akcja                                                             |
| <b>R=0</b>                                                                                                                                                                     | $< 0.5$ | W dół (odcięcie)                                                  |
| <b>R=1, S=1</b>                                                                                                                                                                | $> 0.5$ | W górę (+1 do <b>G</b> )                                          |
| <b>R=1, S=0</b>                                                                                                                                                                | $= 0.5$ | <b>Do parzystej:</b> w górę gdy <b>G=1</b> , w dół gdy <b>G=0</b> |

Własności matematyczne i rzutowanie

|                                              |                                                                                           |
|----------------------------------------------|-------------------------------------------------------------------------------------------|
| Brak łączności                               | $(a + b) + c \neq a + (b + c)$ - zaokrąglenia                                             |
| Brak rozdzielności                           | $a(b + c) \neq ab + ac$                                                                   |
| <b>int</b> $\rightarrow$ <b>float</b>        | możliwa utrata precyzji (zaokrągła)                                                       |
| <b>int</b> $\rightarrow$ <b>double</b>       | bezstratne ( $52 > 32$ bity)                                                              |
| <b>float/double</b> $\rightarrow$ <b>int</b> | obcina do 0; poza zakresem lub <b>NaN</b> $\rightarrow$ <b>TMin</b> ( <b>0x80000000</b> ) |

## 6. Rejestry x86\_64

|                   |                   |                                                       |                   |                   |                                                       |
|-------------------|-------------------|-------------------------------------------------------|-------------------|-------------------|-------------------------------------------------------|
| <code>%rax</code> | <code>%eax</code> | <code>%ax</code><br><code>%ah</code> <code>%al</code> | <code>%rbx</code> | <code>%ebx</code> | <code>%bx</code><br><code>%bh</code> <code>%bl</code> |
| <code>%rcx</code> | <code>%ecx</code> | <code>%cx</code><br><code>%ch</code> <code>%cl</code> | <code>%rdx</code> | <code>%edx</code> | <code>%dx</code><br><code>%dh</code> <code>%dl</code> |
| <code>%rsi</code> | <code>%esi</code> | <code>%si</code><br><code>%sil</code>                 | <code>%rdi</code> | <code>%edi</code> | <code>%di</code><br><code>%dil</code>                 |
| <code>%rbp</code> | <code>%ebp</code> | <code>%bp</code><br><code>%bpl</code>                 | <code>%rsp</code> | <code>%esp</code> | <code>%sp</code><br><code>%spl</code>                 |
| <code>%r8</code>  | <code>%r8d</code> | <code>%r8w</code><br><code>%r8b</code>                | <code>%r9</code>  | <code>%r9d</code> | <code>%r9w</code><br><code>%r9b</code>                |

**Uwaga:** Rejestry od `%r10` do `%r15` używają schematu:  
`%r10`, `%r10d`, `%r10w`, `%r10b`.

### Podział ról rejestrów

|                                                                                                                                 |                                                                                |
|---------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| <code>%rax</code>                                                                                                               | Akumulator. Główny rejestr arytmetyczny. Przechowuje <b>wartość zwracaną</b> . |
| <code>%rdi</code> ,<br><code>%rsi</code> ,<br><code>%rdx</code> ,<br><code>%rcx</code> ,<br><code>%r8</code> , <code>%r9</code> | Kolejno: <b>od 1. do 6. argumentu funkcji</b> . Caller-saved.                  |
| <code>%rsp</code>                                                                                                               | Wskaźnik stosu (SP). Wskazuje wierzchołek ramki.                               |
| <code>%rbp</code>                                                                                                               | Wskaźnik bazy (BP). Początek ramki stosu. Callee-saved.                        |
| <code>%rbx</code> ,<br><code>%r12</code> - <code>%r15</code>                                                                    | Ogólnego przeznaczenia. Wszystkie <b>callee-saved</b> .                        |
| <code>%rip</code>                                                                                                               | Wskaźnik instrukcji (Program Counter).                                         |

### Rejestry wektorowe (zmiennoprzecinkowe)

|                                             |                                                                                                                                         |
|---------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| <code>%xmm0</code> -<br><code>%xmm15</code> | 128-bitowe. <code>%xmm0</code> to wartość zwracana ( <code>float</code> / <code>double</code> ). Pierwsze 8 to argumenty. Caller-saved. |
| <code>%ymm0</code> -<br><code>%ymm15</code> | 256-bitowe rozszerzenie XMM. Dzielią z nimi najmłodsze 128 bitów.                                                                       |

## 7. Tryby adresowania (operandy)

| Tryb                   | Format                      | Obliczanie adresu             |
|------------------------|-----------------------------|-------------------------------|
| Natychmiastowy (Imm)   | <code>\$Imm</code>          | <code>Imm</code>              |
| Rejestrowy (Reg)       | <code>%Ra</code>            | <code>%Ra</code>              |
| Bezpośredni (Mem)      | <code>Imm</code>            | <code>Mem[Imm]</code>         |
| Pośredni (Mem)         | <code>(%Rb)</code>          | <code>Mem[%Rb]</code>         |
| Z przesunięciem        | <code>D(%Rb)</code>         | <code>Mem[%Rb + D]</code>     |
| Skalowany (Ind/scaled) | <code>D(%Rb, %Ri, S)</code> | <code>Mem[%Rb+%Ri*S+D]</code> |
| Skalowany (baseless)   | <code>(, %Ri, S)</code>     | <code>Mem[%Ri * S]</code>     |

**Legenda:** `Imm` / `D` to stała liczbowa, `%Ra` / `%Rb` to rejestr bazowy, `%Ri` to rejestr indeksowy, a `S` to skala (tylko: 1, 2, 4 lub 8).

## 8. Flagi stanu i sterowanie

|           |                                                               |
|-----------|---------------------------------------------------------------|
| <b>ZF</b> | Zero. Wynik to 0 (np. argumenty są równe).                    |
| <b>SF</b> | Sign. Wynik jest ujemny (MSB = 1).                            |
| <b>CF</b> | Carry. Przepełnienie dla liczb <b>bez znaku</b> (unsigned).   |
| <b>OF</b> | Overflow. Przepełnienie dla liczb <b>ze znakiem</b> (signed). |

Sufiksy warunkowe (dla `jX`, `setX`, `cmovX`)

| Sufiks                              | Znaczenie                               |
|-------------------------------------|-----------------------------------------|
| <code>e / z</code>                  | Equal / Zero (równe / wynik to 0)       |
| <code>ne / nz</code>                | Not Equal / Not Zero (nierówne)         |
| <code>s</code>                      | Sign (wynik ujemny, <code>SF=1</code> ) |
| <b>Liczyby ze znakiem (signed)</b>  |                                         |
| <code>g / ge</code>                 | Greater / Greater or Equal              |
| <code>l / le</code>                 | Less / Less or Equal                    |
| <b>Liczyby bez znaku (unsigned)</b> |                                         |
| <code>a / ae</code>                 | Above / Above or Equal (No Carry)       |
| <code>b / be</code>                 | Below / Below or Equal (Carry)          |

### Translacja struktur kontrolnych (C → ASM)

- if-else:** `jX` (skok) lub `cmovX` (liczy obie ścieżki, bez kary za predykcję). `cmovX` odpada przy drogich gałęziach, wyluskaniach (`*p`) i efektach ubocznych (`x++`).
- switch:** tablica skoków → czas  $O(1)$ . Skok pośredni: `jmp *.L4(%rdi,8)`. Brak `break` ⇒ **fall-through**.
- Pętla for:** zawsze redukowana do **while**:  
`init; while(cond) { body; update; }`

### Wzorce asemblerowe dla pętli:

| <code>do-while</code>                         | <code>while</code> (Jump-to-mid)                                                 | <code>while</code> (Guarded)                                                            |
|-----------------------------------------------|----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| <code>loop:</code><br>Body<br>if(T) goto loop | <code>goto test</code><br><code>loop:</code><br>Body<br>test:<br>if(T) goto loop | <code>if(!T) goto done</code><br><code>loop:</code><br>Body<br>if(T) goto loop<br>done: |

## 9. Procedury i stos

Stos rośnie **w dół** (niższe adresy); `%rsp` wskazuje **wierzchołek** (najniższy zajęty adres). `push` zmniejsza `%rsp` o 8, `pop` zwiększa o 8.

- `call dest`: odkłada adres powrotu na stos i skacze do `dest`.
- `ret`: zdejmuje adres powrotu ze stosu i skacze pod niego.

## 10. Konwencja Wywoływań (System V ABI)

`%rdi` → `%rsi` → `%rdx` → `%rcx` → `%r8` → `%r9`

Argumenty 7+: Na stosie od końca. Wynik w `%rax`.

### Rejestry caller-saved vs callee-saved

#### Caller-saved

(Wołający musi zapisać)  
Mogą zostać nadpisane w funkcji.  
By je zachować, caller kładzie je na stos przed `call`.

`%rax`, `%rdi`, `%rsi`,  
`%rdx`, `%rcx`, `%r8` - `%r11`

#### Callee-saved

(Wołany musi przywrócić)  
Muszą zachować stan. Callee musi zapisać je na stos i odtworzyć przed `ret`.

`%rbx`, `%rbp`, `%r12` - `%r15`

Ramka stosu

Każde wywołanie funkcji ma własną ramkę (stąd rekurencja działa naturalnie).

Ramka potrzebna, gdy: brak rejestrów na zmienne (spilling), lokalna tablica/struktura, lub pobrano adres zmiennej (&).

| Prolog                                                                                                                   | Epilog                                                                                          |
|--------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| <pre>pushq %rbp ; zapisz stary base ptr movq %rsp, %rbp ; nowy base ptr = rsp subq \$32, %rsp ; alokacja 32 bajtów</pre> | <pre>addq \$32, %rsp ; zwolnij alokację popq %rbp ; przywróć base ptr ret ; skok powrotny</pre> |

leave : zastępuje movq %rbp, %rsp + popq %rbp jedną instrukcją.

11. Architektura CPU

Fazy przetwarzania instrukcji

Fetch → Decode → Execute → Memory → Write

Pipelining i hazardy

Nakładanie faz zwiększa throughput, ale rodzi konflikty (hazardy):

|            |                                                                                                                                                                                               |
|------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Danych     | Odczyt po zapisie - wynik jeszcze nie jest w rejestrze, a kolejna instrukcja już go potrzebuje.<br><b>Rozwiązanie:</b> forwarding/bypassing (wynik z ALU prosto na wejście) lub stall/bubble. |
| Sterowania | Skok wymusza odgadnięcie następnego adresu.<br><b>Rozwiązanie:</b> branch prediction. Pomyłka → czyszczenie potoku (misprediction penalty).                                                   |

Out-of-Order

- Nowoczesne procesory nie wykonują instrukcji sekwencyjnie:
- Superskalarność:** wiele instrukcji na cykl (wiele jednostek wykonawczych).
  - Register renaming:** mapowanie rejestrów logicznych (%rax) na wiele fizycznych - eliminuje fałszywe zależności.
  - Reorder buffer:** instrukcje liczone asynchronicznie, ale zatwierdzane w kolejności programu (spójność).

Miary wydajności

|                  |                                                                                                                                           |
|------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Latency bound    | Limit wynikający z łańcucha zależności danych. Zależy od opóźnienia jednostki. Np. wynik z poprzedniej iteracji jest potrzebny w obecnej. |
| Throughput bound | Limit wynikający z przepustowości sprzętu.                                                                                                |

Fazy przetwarzania

|            |                                                                                                                            |
|------------|----------------------------------------------------------------------------------------------------------------------------|
| Dispatch   | Dekodowanie i alokacja w stacji rezerwacyjnej. CPU może zlecić ograniczoną liczbę instrukcji na cykl.                      |
| Execute    | Wykonywanie właściwe. Zależne instrukcje mogą rozpocząć e w cyklu udostępnienia wyniku (u) przez instrukcję poprzedzającą. |
| Write-back | Udostępnianie wyniku na magistrali. W tym samym cyklu zależne instrukcje zaczynają e.                                      |
| Retire     | Zatwierdzenie stanu architektury. Musi zachodzić ściśle in-order.                                                          |

12. Tablice i struktury

Reprezentacja tablic w pamięci

| Typ            | Deklaracja | Adres                                                                        |
|----------------|------------|------------------------------------------------------------------------------|
| Jednowymiarowa | T A[IL]    | $A[i] = x_A + i \times \text{sizeof}(T)$                                     |
| Wielowymiarowa | T A[R][C]  | $A[i][j] = x_A + (i \times C + j) \times \text{sizeof}(T)$                   |
| Wielopoziomowa | T *A[LL]   | $M[x_A + i \times 8] + j \times \text{sizeof}(T)$<br>(dwa odczyty z pamięci) |

Wyrównanie danych i padding

- Zasada ogólna:** obiekt rozmiaru K leży pod adresem podzielny przez K.
- Struktury:** każde pole wyrównane do własnego K; rozmiar całości podzielny przez największe wyrównanie (padding na końcu).
- Optymalizacja:** pola od największego do najmniejszego → minimalny padding.

13. Układ pamięci

Mapa pamięci procesu

|             |                                                                |
|-------------|----------------------------------------------------------------|
| Stack ↓     | Rośnie w dół. Zmienne lokalne, adresy powrotu (limit 8MB).     |
| Shared libs | Współdzielone biblioteki (np. libc.so).                        |
| Sterna ↑    | Rośnie w górę. Dynamiczna alokacja (malloc, new).              |
| Data        | Zmienne globalne i statyczne (zainicjowane i niezainicjowane). |
| Text        | Read-only. Binarny kod wykonywalny programu.                   |

Góra tabeli = wysokie adresy, dół = niskie adresy.

Zagrożenia i mechanizmy obronne

|                 |                                                                                                                 |
|-----------------|-----------------------------------------------------------------------------------------------------------------|
| Buffer overflow | Brak sprawdzania granic tablic. Nadpisuje stos (stary %rbp, adres powrotu) → przejęcie sterowania.              |
| ROP (Gadżety)   | Return-Oriented Programming. Sklejanie legalnych skrawków kodu zakończonych ret - omija zakaz wykonywania (NX). |
| Stack canaries  | Losowa wartość tuż przed adresem powrotu. Przed ret weryfikacja (xor + je); uszkodzona → abort().               |
| NX / ASLR       | NX: zakaz wykonywania kodu ze stosu i sterty.<br>ASLR: losowanie bazowych adresów obszarów przy każdym starcie. |

Unie

Wszystkie pola współdziela ten sam adres (offset 0); rozmiar = największe pole. Służą do manipulacji bitowych z ominięciem systemu typów (np. bity float jako int).

## 14. Optymalizacje i ich ograniczenia

Kompilator nie zoptymalizuje kodu, jeśli mogłoby to zmienić zachowanie programu (choćby dla specyficznych argumentów).

| Optymalizacyjne blokady |                                                                                                                                                                         |
|-------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Aliasing pamięci        | <code>*p</code> i <code>*q</code> mogą wskazywać na to samo $\rightarrow$ wymuszony odczyt/zapis do RAM zamiast rejestru. Pomaga słowo kluczowe <code>restrict</code> . |
| Wywołania funkcji       | Funkcja w pętli może mieć ukryte efekty uboczne - nie zostanie wyrzucona poza pętlę. <b>Rozwiązanie:</b> inlining lub makra.                                            |
| Arytmetyka float        | Brak łączności: $(a + b) + c \neq a + (b + c)$ .<br>Kompilator <b>nigdy</b> nie zmieni kolejności działań (troska o precyzję).                                          |

| Podstawowe techniki optymalizacji |                                                                                                                                                   |
|-----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| Code motion                       | Wyrzucenie niezmienników (np. <code>x * y</code> ) poza pętlę.                                                                                    |
| Strength reduction                | Kosztowne $\rightarrow$ tańsze (np. <code>x * 64</code> $\rightarrow$ <code>x &lt;&lt; 6</code> , iteracja wskaźnikiem zamiast mnożenia indeksu). |
| Share subexpr.                    | Jednokrotne obliczanie powtarzających się podwyrażeń.                                                                                             |
| Loop unrolling                    | Rozwinięcie pętli (np. krok co 2). Mniejszy narzut pętli, więcej równoległości dla CPU.                                                           |

## 15. Katalog instrukcji (AT&T)

| Opcode                      | Efekt                         | Opis                                                   |
|-----------------------------|-------------------------------|--------------------------------------------------------|
| <code>mov S, D</code>       | $D \leftarrow S$              | Kopiuje wartość z S do D.                              |
| <code>lea S, D</code>       | $D \leftarrow \text{addr}(S)$ | Oblicza adres S (bez czytania pamięci).                |
| <code>add S, D</code>       | $D \leftarrow D + S$          | Dodawanie (ustawia flagi).                             |
| <code>sub S, D</code>       | $D \leftarrow D - S$          | Odejmowanie (ustawia flagi).                           |
| <code>imul S, D</code>      | $D \leftarrow D * S$          | Mnożenie liczb ze znakiem.                             |
| <code>sal / shl k, D</code> | $D \ll= k$                    | Przesunięcie w lewo (mnożenie przez $2^k$ ).           |
| <code>sar k, D</code>       | $D \gg= k$                    | Arytmetyczne w prawo (dzielenie). <b>Kopiuje znak.</b> |
| <code>shr k, D</code>       | $D \gg= k$                    | Logiczne w prawo. Dopełnia zerami.                     |
| <code>and / or S, D</code>  | $D \leftarrow D \& /   S$     | Operacje bitowe (ustawiają flagi).                     |
| <code>xor S, D</code>       | $D \leftarrow D \wedge S$     | Często jako <code>xor %rax, %rax</code> do zerowania.  |

| Porównania i sterowanie  |                                                    |                                                     |
|--------------------------|----------------------------------------------------|-----------------------------------------------------|
| <code>cmp S1, S2</code>  | $S2 - S1$                                          | Ustawia flagi jak odejmowanie, nie zapisuje wyniku. |
| <code>test S1, S2</code> | $S2 \& S1$                                         | Ustawia flagi jak <b>AND</b> .                      |
| <code>jX dest</code>     | $\text{if}(X) \text{\%rip} \leftarrow \text{dest}$ | Skok warunkowy (X = warunek).                       |
| <code>setX D</code>      | $\text{if}(X) D \leftarrow 1$                      | Ustawia najmłodszy bajt na 0 lub 1 wg flag.         |
| <code>cmoveX S, D</code> | $\text{if}(X) D \leftarrow S$                      | Warunkowe kopiowanie (zamiast skoku).               |

| Stos i zarządzanie procedurami |                                                                      |                                                       |
|--------------------------------|----------------------------------------------------------------------|-------------------------------------------------------|
| <code>push S</code>            | $\text{\%rsp} - = 8$<br>$M[\text{\%rsp}] \leftarrow S$               | Odkłada na stos (zmniejsza <code>%rsp</code> ).       |
| <code>pop D</code>             | $D \leftarrow M[\text{\%rsp}]$<br>$\text{\%rsp} + = 8$               | Zdejmuje ze stosu do D (zwiększa <code>%rsp</code> ). |
| <code>call dest</code>         | $\text{push } \text{\%rip}$<br>$\text{\%rip} \leftarrow \text{dest}$ | Skok do funkcji (zapisuje adres powrotu na stosie).   |
| <code>ret</code>               | $\text{pop } \text{\%rip}$                                           | Zdejmuje adres powrotu ze stosu i skacze pod niego.   |
| <code>leave</code>             | $\text{\%rsp} \leftarrow \text{\%rbp}$<br>$\text{pop } \text{\%rbp}$ | Sprząta ramkę stosu (odwrotność prologu).             |

Operacje wektorowe

Rozszerzenie AVX. Przedrostek `v` (nie niszczy źródeł).

|                                    |                            |                                                                                                                   |
|------------------------------------|----------------------------|-------------------------------------------------------------------------------------------------------------------|
| <code>vmovaps / ups S, D</code>    | $D \leftarrow S$           | Kopiuje wektor. <code>a</code> (aligned - szybkie, adres podzielny przez wyrównanie), <code>u</code> (unaligned). |
| <code>vaddps / pd S1, S2, D</code> | $D \leftarrow S2 + S1$     | Dodawanie wielokrotne (packed). <code>ps</code> (single-float), <code>pd</code> (double).                         |
| <code>vmulps / pd S1, S2, D</code> | $D \leftarrow S2 * S1$     | Mnożenie wektorowe.                                                                                               |
| <code>vfmadd231ps S1, S2, D</code> | $D \leftarrow S2 * S1 + D$ | <b>Fused Multiply-Add.</b> Mnoży i dodaje do <code>D</code> w jednym kroku.                                       |

Operacje zmiennoprzecinkowe

|                                   |                                |                                                                                                 |
|-----------------------------------|--------------------------------|-------------------------------------------------------------------------------------------------|
| <code>movsd / movss S, D</code>   | $D \leftarrow S$               | Kopiuje skalar ( <code>double</code> / <code>float</code> ) między rejestrami XMM lub pamięcią. |
| <code>movapd / movaps S, D</code> | $D \leftarrow S$               | Kopiuje wyrównany wektor zmiennoprzecinkowy.                                                    |
| <code>addsd / subsd S, D</code>   | $D \leftarrow D \text{ op } S$ | Skalarne dodawanie / odejmowanie podwójnej precyzji.                                            |
| <code>xorpd / xorps S, D</code>   | $D \leftarrow D \hat{\ } S$    | Bitowy XOR na XMM. Jako <code>xorpd %xmm0, %xmm0</code> zeruje rejestr.                         |
| <code>ucomisd S1, S2</code>       | $S2 - S1$                      | Porównuje <code>double</code> i ustawia flagi <code>ZF, PF, CF</code> w <code>%rflags</code> .  |

**Uwaga o sufiksach:** Instrukcje przyjmują sufiks określający rozmiar danych: `b` (8-bit), `w` (16-bit), `l` (32-bit), `q` (64-bit).  
Np. `movl` kopiuje 32 bity (i automatycznie zeruje górną połowę 64-bitowego rejestru!).